Aho-Corasick PatternGuard Project Documentation

This document provides the architecture overview, algorithm explanation, assumptions made, challenges faced, and future improvements for the PatternGuard Spam & Content Detector project built using the Aho-Corasick algorithm.

1. Architecture Overview

Frontend: The frontend is developed using HTML, CSS, and JavaScript. It provides a user-friendly interface where users can add keywords, load presets, input text, and view highlighted results.

Backend: The backend is developed using FastAPI in Python. It exposes REST API endpoints for adding keywords, clearing keywords, checking server health, and searching text.

Algorithm Layer: The Aho-Corasick algorithm is implemented using a Trie data structure with failure links to efficiently search multiple patterns in a single pass.

Data Flow: The frontend sends user keywords and text to the FastAPI backend through HTTP requests. The backend processes the request using the Aho-Corasick automaton and returns matching results.

2. Algorithm Explanation

The Aho-Corasick algorithm is a multi-pattern string matching algorithm used for efficiently finding multiple keywords in a text simultaneously.

Step 1: Build a Trie
All keywords are inserted into a Trie where each node represents a character.

Step 2: Create Failure Links
Failure links are created using Breadth-First Search (BFS). These links allow the algorithm to jump to the next best matching state when a mismatch occurs.

Step 3: Search Text
The algorithm scans the text character by character only once. Whenever a pattern is matched, it records the keyword and position.

Time Complexity:
- Building Trie: O(total length of keywords)
- Searching Text: O(text length + number of matches)
- Highly efficient compared to naive searching.

3. Assumptions Made

- Input keywords are converted to lowercase for case-insensitive matching.
- Duplicate keywords are ignored.

- The backend server runs locally on port 8000.
- Users provide valid text input before scanning.
- The frontend and backend run on the same machine during development.

4. Challenges Faced

- Understanding and implementing failure links correctly.
- Handling overlapping keyword matches such as 'he' inside 'she'.
- Managing frontend-backend communication using FastAPI and JavaScript fetch requests.
- Debugging CORS issues and API connection problems.
- Ensuring the UI updates dynamically without page refresh.

5. Future Improvements

- Deploy the backend on cloud platforms such as Render or Railway.
- Add database integration for storing keywords and scan history.
- Implement authentication and user accounts.
- Support PDF and document uploads for scanning.
- Improve visualization of matches and analytics.
- Add real-time scanning and AI-based spam classification.

6. Conclusion

The PatternGuard project demonstrates the practical implementation of the Aho-Corasick algorithm for fast and efficient multi-pattern matching. The system successfully combines a modern frontend interface with a powerful backend search engine to detect spam, phishing, profanity, and other keywords in real time.